

# Software Architecture

---

## – Logical Layers and Physical Tiers

It is important to understand the distinction between tiers and layers:

- **Tiers:** Refer to the physical or logical separation of the system into distinct segments (e.g., Client, Business Logic, Integration, Data).
- **Layers:** Refer to sub-divisions within a tier that further separate functionality.

### Tier 1 – Client / Presentation Tier

- **Logical Layer:** Presentation Layer
- **Purpose:** Responsible for user interaction, displaying information, capturing user input, and performing client-side validations. It provides the UI for both Internet and Mobile Banking.
- **Justification:** Separating this tier allows different client technologies (web browsers, iOS, Android) to be developed independently and scaled based on access patterns, without affecting core business logic. This directly addresses the requirement for both Internet and Mobile Banking interfaces.

### Tier 2 – Application / Business Logic Tier

- **Logical Layers:** Business Logic Layer, Service Layer
- **Purpose:** Houses the core business rules and functionalities (e.g., funds transfer logic, account summary, special service request initiation). It acts as a unified API to the Presentation Tier and handles application-level security (authentication and authorisation). It ensures business logic is common and shared for both web and mobile clients.
- **Justification:** Centralising business logic ensures consistency across all digital channels, promotes reusability, simplifies maintenance, and creates a crucial security boundary between public-facing clients and sensitive backend systems.

### Tier 3 – Integration Tier

- **Logical Layers:** Integration Layer, Data Access Layer (for external systems)
- **Purpose:** Acts as a mediation and translation layer between the Business Logic Tier and existing backend systems. It handles protocol conversions, data transformations, message routing, and ensures secure communication with heterogeneous interfaces (REST, ISO 8583, SOAP, POX). It also facilitates asynchronous communication for workflows and notifications.
- **Justification:** This tier is critical due to SriBank's existing heterogeneous systems. It decouples core business logic from the complexities of legacy system interfaces, making the overall architecture more resilient to changes and easier to manage.

### Tier 4 – Data Tier

- **Logical Layer:** Data Storage Layer

- **Purpose:** Manages the persistence of data specific to the new platform (user preferences, session data, audit logs, workflow states) and serves as the physical location of external databases accessed via the Integration Tier.
- **Justification:** Separating the Data Tier provides specialised management of data, allowing for optimised database performance, robust transaction management (ACID properties), data security, and independent scaling of storage resources.

## – Key Components of Each Tier

### Tier 1: Client / Presentation Tier

- **Internet Banking Web Application:** Built using a modern JS framework (React, Angular, or Vue.js) as a Single Page Application (SPA). Includes client-side validation logic, HTML/CSS/JavaScript assets, and an API client library to interact with backend RESTful APIs.
- **Mobile Banking App (iOS & Android):** Developed using Swift/Objective-C (iOS) or Kotlin/Java (Android), or cross-platform using Flutter. Includes API client library, Push Notification Handler (APNS/FCM), and local data storage for caching.

### Tier 2: Application / Business Logic Tier

- **API Gateway:** Entry point for all client requests. Handles request routing, authentication and authorisation (including 2FA), rate limiting/throttling, and logging/monitoring.
- **Retail Banking Service:** Implements core functions: Balance Inquiry, Account Summary, Funds Transfer, Utility Bill Payments.
- **Account Management Service:** Handles online self-registration and customer profile management.
- **Special Service Request Service:** Initiates workflow-based requests such as Loan Requests, Credit Limit Increments, and new account creation. Publishes events to the message broker.
- **Security & Identity Service:** Manages user registration, password policies, 2FA enrolment/verification, session management, and JWT token issuance.
- **Notification Orchestration Service:** Exposes an API for other internal services to trigger push or SMS notifications.

### Tier 3: Integration Tier

- **Message Broker (e.g., Kafka or RabbitMQ):** Manages topics/queues for asynchronous communication such as notification events, workflow request events, and workflow status updates.
- **Core Banking System Adapter:** Translates internal requests into RESTful calls for the Core Banking System and maps responses back to the internal format.
- **Inter-bank Payment Gateway Adapter:** Formats internal payment requests into ISO 8583 messages for the Central Bank's gateway and handles parsing of responses.
- **Credit Card System Adapter:** Handles SOAP message construction and parsing for communication with the Credit Card System.
- **Internal Payment Switch Adapter:** Manages TCP/IP-based POX communication and data mapping for MasterCard/VISA processing.

- **Workflow Engine:** Manages the state and flow of complex multi-step special service requests (e.g., loan approvals, new account creation), assigning tasks to relevant departments.
- **Notification Delivery Service:** Interfaces with APNS (Apple) and FCM (Google) for push notifications, and with an SMS gateway for text messages. Includes a notification templating engine.

#### Tier 4: Data Tier

- **Relational Database (RDBMS):** Stores data specific to the new platform — user digital profiles, notification preferences, audit trails for digital transactions, workflow request states, and user session data.
- **External System Databases (accessed via Integration Tier):** Core Banking System (KYC, account balances, transaction history), Credit Card System (credit limits, billing), Inter-bank Payment Gateway, and Internal Payment Switch.

### – Architectural Patterns

#### 1. N-Tier Architecture

The N-Tier (specifically 4-Tier) pattern logically and physically separates the application into distinct layers, each with a specific responsibility.

- **Scalability:** Each tier can be independently scaled. If mobile app usage surges, only the Presentation Tier needs scaling — not the entire Data Tier.
- **Maintainability:** Changes in one tier have minimal impact on others. Updating the UI does not require changes to core business logic.
- **Security:** The Business Logic Tier acts as a security boundary (DMZ), preventing direct client access to sensitive data and legacy systems.
- **Flexibility:** Different technologies can be used per tier — native mobile apps, Java/Spring Boot backend, and a high-performance RDBMS for data.

#### 2. Service-Oriented Architecture (SOA)

SOA structures the application as a collection of loosely coupled, reusable services, each encapsulating a specific business capability exposed via a well-defined interface.

- **Integration:** Creates specialised service adapters that abstract the complexity of heterogeneous protocols (REST, SOAP, ISO 8583, POX). Business logic interacts with a generic service without needing to know the backend protocol.
- **Reusability:** Core banking functions (FundsTransfer, AccountSummary, Authentication) are reusable across both Internet and Mobile Banking, and can be used by future systems within SriBank.
- **Maintainability:** Services can be updated independently. Changing Utility Bill Payments logic requires only that service's redeployment.
- **Scalability:** High-demand services like Balance Inquiry can be scaled independently from low-traffic ones like Loan Requests.

#### 3. Publish-Subscribe (Pub/Sub) Architecture

In the Pub/Sub pattern, publishers send messages to a message broker without knowing the recipients. Subscribers express interest in specific topics and receive relevant messages asynchronously.

- **Reliable Notifications:** When a transaction completes, the core banking service publishes a `TransactionCompletedEvent`. The Notification Delivery Service picks this up asynchronously, ensuring the notification is delivered without blocking the transaction.
- **Asynchronous Workflows:** Special service requests (loans, credit limit increments) are not real-time. The Workflow Engine subscribes to request events and manages multi-departmental approvals independently.
- **Scalability:** The Message Broker buffers notification spikes. Multiple Notification Service instances can process messages in parallel.
- **Decoupling:** Publishers do not need to know how notifications are delivered (APNS, FCM, SMS). New notification channels can be added by simply adding a new subscriber.

#### 4. Model-View-Controller (MVC) — Client-Side Focus

MVC separates an application into three components: the Model (data/business rules), the View (UI), and the Controller (handles user input and orchestrates updates).

- **User Responsiveness:** The Controller manages user input locally, updating the View dynamically without full page reloads, making the banking app feel fast and fluid.
- **Client-Side Validation:** The Model contains validation rules checked instantly in the browser before any server call — e.g., verifying account number format or ensuring amounts are positive.
- **Maintainability:** The clear separation of concerns makes front-end code easier to understand and maintain. Developers can work on UI (View) independently from data (Model) or logic (Controller).
- **Testability:** Each MVC component can be unit-tested independently, improving code quality.

#### – Architecture Diagram Note

The architecture diagram should illustrate all 4 tiers, components described in Part (b), and communication flows reflecting the 4 architectural patterns discussed in Part (c). Marks are awarded based on component coverage and pattern representation.

## Client-Side Architecture

---

#### – Most Suitable Client-Side Architectural Pattern

The most suitable client-side architectural pattern for the browser-based Internet Banking application is Client-Side Model-View-Controller (MVC), or its modern variations such as Model-View-ViewModel (MVVM) or Model-View-Presenter (MVP). This pattern provides a structured approach for building interactive Single Page Applications (SPAs) that deliver rich user experiences with strong client-side validation.

In this context:

- **Model:** Manages application data and contains validation rules (e.g., valid account number formats, positive transfer amounts, correct date ranges).

- **View:** Renders the user interface and displays feedback such as error messages or success confirmations, reacting to changes in the Model.
- **Controller:** Receives user input from the View, invokes validation in the Model, and decides whether to update the View (show error) or proceed to call the backend API.

## – Client-Side Validation and User Experience

### Enabling Client-Side Validation

Client-Side MVC enables immediate, in-browser validation through the collaboration of all three components:

- **Model-Driven Validation:** The Model holds validation rules directly linked to the data it manages. For example, a Transaction Model enforces that amounts must be positive and account numbers must match a specific format (e.g., 10-digit numeric).
- **Controller as Orchestrator:** When the user types in an input field, the Controller intercepts the input. Before attempting any server call or Model update, it checks the Model's validation rules.
- **View for Instant Feedback:** If validation fails, the Controller instructs the View to display contextual error messages next to the offending fields — no network delay, no page reload.

Example — Funds Transfer:

- **View:** Displays input fields for 'Beneficiary Account Number' and 'Transfer Amount'.
- **Controller:** On each input event, retrieves the value and asks the Model to validate.
- **Model:** Contains regex patterns for account numbers and constraints for amounts (e.g., not zero, not negative).
- **View (Update):** If validation fails, immediately displays a red border and message: "Invalid Account Number Format" or "Amount must be greater than zero" — all within the browser, without any server call.

### Contributing to Better User Responsiveness and Experience

- **Reduced Server Round-Trips:** After the initial SPA load, subsequent user actions only require small JSON data exchanges with the backend. The View re-renders only the changed parts of the page, eliminating full page reloads.
- **Fluid, App-Like Navigation:** Users navigate between sections (e.g., Dashboard, Funds Transfer, Account History) without waiting for the server to generate new pages. Transitions feel instantaneous.
- **Consistent State Management:** The Model centralises application state. After a funds transfer, the Model updates the balance, and all Views displaying that balance (Dashboard, Account Summary) automatically re-render to reflect the new value.

Example — Account Summary:

- User clicks 'View Transactions' for an account.
- Controller fetches only the transaction data (not the full page) from the API.
- Model updates with the transaction list.

- View dynamically replaces only the transaction list section with smooth animation — the rest of the page stays unchanged.

## – Architectural Considerations and Trade-offs in a Banking Context

### 1. Security Implications

- **Client-side code is inspectable:** Any JavaScript running in the browser can be viewed and manipulated by users. All client-side validation must be duplicated and enforced server-side — the client-side validation is only for UX, never for security.
- **API Security is critical:** The SPA relies entirely on backend APIs. Robust security measures are essential: short-lived JWT tokens, HTTP-only cookies for refresh tokens, and strict HTTPS enforcement. Sensitive data must never be stored in localStorage or sessionStorage as these are vulnerable to XSS attacks.
- **Partial exposure of logic:** Presentation logic and validation rules embedded in client-side code are visible to anyone who inspects the browser source. No truly sensitive business logic (e.g., credit scoring, transaction limits) should ever reside exclusively on the client.

### 2. Initial Load Times

- **Larger Initial Bundle:** The entire SPA framework, Model, View, Controller code, and dependencies must be downloaded on the first visit. This can result in a slower 'time to interactive' compared to traditional server-rendered pages.
- **Mitigation Strategies:** Code splitting (loading only the required module for the current page), lazy loading of non-critical components, tree shaking (removing unused code), and aggressive caching of static assets are essential to reduce perceived load time.

### 3. Increased Client-Side Complexity

- **Higher Development Skill Requirements:** Building well-structured client-side MVC/MVVM applications requires more advanced JavaScript expertise and a deeper understanding of front-end architecture patterns.
- **Debugging Challenges:** Issues spanning multiple MVC layers (e.g., data binding problems, unexpected View updates) can be harder to trace than debugging simple server-rendered pages.
- **State Management Complexity:** As the banking SPA grows (more services, more views), managing global application state (e.g., user session, current account context) requires careful architecture, often addressed with state management libraries like Redux or Vuex.

Despite these trade-offs, Client-Side MVC remains the most suitable choice for a modern, customer-centric Internet Banking platform that prioritises user responsiveness and a rich interactive experience — provided all architectural considerations, particularly security, are rigorously addressed on both the client and server sides.

## – Integration Architecture for Heterogeneous Systems

---

## – Primary Architectural Pattern for Integration

Given SriBank PLC's scenario with four existing systems using completely different interfaces (REST, ISO 8583, SOAP, TCP/IP POX), the primary architectural pattern most suitable for handling these integrations is a dedicated Integration Layer, implemented using Enterprise Integration Patterns (EIPs). This is typically manifested through an Enterprise Service Bus (ESB) or a more modern API Management Platform with Integration Microservices.

### What is an Integration Layer?

An Integration Layer is a dedicated architectural segment that facilitates communication and data exchange between disparate applications and systems. It acts as a central nervous system for enterprise-wide data flow. It abstracts the complexities of individual system interfaces, allowing the core business logic to interact with external systems through a unified, standardised mechanism.

### Justification

- **Handling Heterogeneity:** SriBank's systems use vastly different protocols and message formats. Without an Integration Layer, direct point-to-point connections from the banking platform to each system would create a 'spaghetti architecture' — a tangled, brittle mess. The Integration Layer uses specialised adapters to convert internal requests into each system's specific protocol.
- **Decoupling:** The Business Logic Tier interacts with a stable internal interface exposed by the Integration Layer. If an external system's API changes (e.g., a field name in the Core Banking REST API or an ISO 8583 specification update), only the corresponding adapter needs updating — not the entire business logic.
- **Centralised Management:** Provides a single point for managing all external system connections, enabling centralised logging, monitoring, and error handling across all integration flows.
- **Security Enhancement:** The Integration Layer acts as a security enforcement point, handling authentication, authorisation, and encryption specific to each external system. It abstracts sensitive credentials away from the Business Logic Tier.
- **Reusability:** Integration components (e.g., the Core Banking Adapter) become reusable assets. Any part of the new platform, or even future systems within SriBank, can leverage these pre-built, tested components.

## – Three Enterprise Integration Patterns (EIPs)

### EIP 1: Splitter

The Splitter pattern takes a single composite message and breaks it down into multiple smaller, individual messages that can be processed independently. This is essential when a single user action requires multiple sub-operations across different systems.

Application — Utility Bill Payment for Multiple Bills:

- **Problem:** A user wants to pay electricity, water, and phone bills in a single action. The Inter-bank Payment Gateway only accepts one bill payment per request.



- The Retail Banking Service receives a composite Multi-Bill Payment Request containing a list of bills.
- A Splitter component within the Integration Tier breaks this into 3 individual Single Bill Payment Request messages.
- Each individual message is routed to the Inter-bank Payment Gateway Adapter independently.
- Benefit: Each bill is processed as a distinct transaction. If one payment fails, it does not block the others, improving fault tolerance and enabling granular status tracking per payment.

## EIP 2: Content-Based Router

The Content-Based Router examines the content (payload or headers) of an incoming message and, based on predefined rules, routes it to one or more specific endpoints or processing channels. This is critical when different types of messages require different handling.

Application — Funds Transfer Routing:

- Problem: A Funds Transfer request could target an internal SriBank account, a local Sri Lankan bank account, or an international account — each requiring a completely different backend system.
- The Integration Tier's Content-Based Router receives the TransferRequest and examines fields like toAccountBankIdentifier or transferType.
- **Routing Rules:** If internal SriBank account → route to Core Banking Adapter. If local external bank → route to Inter-bank Payment Gateway Adapter (ISO 8583). If international → route to Internal Payment Switch Adapter (TCP/IP POX).
- Benefit: Each type of transfer is directed to the correct system. The Business Logic Tier simply sends a generic TransferRequest and the router handles the complexity, maintaining modularity and efficiency.

## EIP 3: Aggregator

The Aggregator pattern collects a set of related messages and combines them into a single composite message. It is often used in conjunction with a Splitter or Scatter-Gather pattern, where a request is fanned out to multiple systems and their individual responses need to be gathered and merged before returning a single result. The Aggregator uses a Correlation ID to link related messages and a completion condition to know when all responses have arrived.

Application — Account Summary with Credit Card Details:

- Problem: A user requests an Account Summary that includes both deposit account balance (from Core Banking System) and credit card balance/limit (from Credit Card System). These are two separate systems requiring two separate queries.
- The Retail Banking Service initiates two parallel sub-requests: one to the Core Banking Adapter for deposit account details, and one to the Credit Card System Adapter for credit card information.
- Both sub-requests are tagged with the same Correlation ID linking them to the original GetAccountSummaryRequest.



- The Aggregator waits for responses from both systems (completion condition: 2 responses received).
- Once both responses arrive, the Aggregator merges them into a single comprehensive AccountSummaryResponse containing data from both systems.
- Benefit: The user sees a unified, complete account summary in one response. Parallel querying reduces total wait time, and the complexity of multi-system integration is completely hidden from the client.

| EIP Pattern          | Use Case in SriBank                                   | Key Benefit                              |
|----------------------|-------------------------------------------------------|------------------------------------------|
| Splitter             | Split multi-bill payment into individual transactions | Fault isolation, granular tracking       |
| Content-Based Router | Route funds transfer to correct backend system        | Modularity, clean separation of concerns |
| Aggregator           | Combine account + credit card data into one summary   | Unified user view, parallel processing   |

## Asynchronous Notification Functionality

### – Why Asynchronous Communication is Appropriate for Notifications

#### Understanding Synchronous vs. Asynchronous Communication

| Aspect         | Synchronous                                                | Asynchronous                                                |
|----------------|------------------------------------------------------------|-------------------------------------------------------------|
| Nature         | Sender waits for an immediate response before continuing   | Sender fires and moves on; no waiting                       |
| Analogy        | A phone call — you wait for the other person to respond    | Sending an email — you send it and continue your work       |
| Blocking       | Yes — the main process is blocked until a response arrives | No — the main process continues immediately                 |
| Coupling       | Tight — sender depends on receiver's availability          | Loose — sender and receiver operate independently           |
| Failure Impact | Receiver failure can cause sender failure or rollback      | Receiver failure does not affect the sender                 |
| Scalability    | Limited — each notification adds latency to the main flow  | High — notifications are queued and processed independently |

### Why Asynchronous is the Right Choice for SriBank's Notifications

Consider the scenario of a Funds Transfer Completion:

- **User Expectation:** The user expects an immediate confirmation of their funds transfer on screen. This confirmation must not be delayed because their phone might be offline or the push notification service is having issues.
- **Platform Reliability:** The banking platform's primary goal is to accurately process financial transactions. If the notification component fails, the core transaction must not fail or roll back.
- **Scalability:** During peak hours, thousands of transactions might complete simultaneously, each requiring a notification. A synchronous approach would create immense pressure on the notification delivery system, potentially causing cascading failures across the entire platform.

With asynchronous communication, the bank's core system publishes a `TransactionCompletedEvent` and immediately returns the transaction confirmation to the user. A separate Notification Service independently picks up this event and handles push notification/SMS delivery. If the mobile network is congested or the device is offline, the notification service retries later — without affecting the user experience on the banking platform.

## – Most Suitable Architectural Pattern: Publish-Subscribe (Pub/Sub)

The Publish-Subscribe (Pub/Sub) architectural pattern is the most suitable approach for implementing asynchronous notification functionality for SriBank PLC. It perfectly addresses the need for decoupled, scalable, and reliable notification delivery.

### How Pub/Sub Works

- **Publishers:** Components that generate events (e.g., Retail Banking Service publishing a `TransactionCompletedEvent`). Publishers send messages to specific topics on the Message Broker without knowing who will consume them.
- **Message Broker (e.g., Kafka, RabbitMQ):** The central component that receives, stores, and routes messages. It persists messages to disk for reliability and distributes them to all interested subscribers.
- **Subscribers:** Components that express interest in specific topics (e.g., Notification Delivery Service subscribes to `TransactionCompletedEvent` topics). They receive and process messages independently of publishers.

### How Pub/Sub Addresses SriBank's Notification Needs

- **Asynchronous Decoupling:** When a transaction completes, the Retail Banking Service publishes a `TransactionCompletedEvent` to the message broker and immediately finishes its work. The Notification Delivery Service picks this up independently and handles the actual push/SMS delivery. Core banking is never blocked by notification processing.
- **Reliable Guaranteed Delivery:** The broker persists messages to disk before acknowledging receipt. Even if the Notification Delivery Service is temporarily down, messages queue up safely in the broker and are processed once the service recovers.
- **Scalability:** The Notification Delivery Service can be horizontally scaled — multiple instances can consume messages in parallel from the broker, increasing throughput during peak hours.

- **Flexibility for Future Channels:** Currently, SriBank requires push notifications and SMS. To add email alerts in the future, a new EmailNotificationService subscriber is deployed. Publishers require zero modification.

## – Ensuring Reliable and Scalable Delivery Under Volatile Conditions

### Ensuring Reliability

- **Decoupling (Time and Space):** Publishers and subscribers operate independently. If the Notification Delivery Service is unavailable (e.g., during deployment or an external push gateway outage), the core banking services are not blocked. Messages accumulate in the broker and are processed once the service recovers — no notifications are lost.
- **Message Persistence and Durability:** Message brokers like Kafka and RabbitMQ persist messages to disk immediately upon receipt, before acknowledging the publisher. Even if the broker itself crashes and restarts, messages are not lost. This ensures 'at-least-once' delivery: a message is only removed from the broker once the subscriber explicitly acknowledges successful processing.
- **Acknowledgement Mechanism:** If the Notification Delivery Service processes a message but fails before sending an acknowledgement, the broker assumes delivery failed and redelivers the message to another available instance. This prevents silent message loss.
- **Dead-Letter Queues (DLQs):** If a message fails processing after a configured number of retry attempts (e.g., due to an invalid message format or a persistent external API error), it is moved to a Dead-Letter Queue instead of being discarded. Operations teams can inspect, fix, and reprocess these messages, preventing data loss and enabling post-incident recovery.

### Ensuring Scalability

- **Asynchronous Load Smoothing:** The Message Broker acts as a buffer. During peak transaction periods, messages accumulate in the broker and are processed at the Notification Service's sustainable rate. This prevents notification processing from overwhelming either the core banking system or the external push gateways.
- **Horizontal Scaling of Subscribers:** The Notification Delivery Service can be deployed as multiple instances. The message broker automatically distributes incoming messages among all available instances using consumer groups, enabling parallel processing. As notification volume grows, more instances are added with no architectural changes required.
- **High Availability Through Redundancy:** If one Notification Delivery Service instance fails, the broker automatically reassigns its pending messages to other healthy instances. This ensures continuous processing and high availability, even during partial system failures.
- **Independent Evolution:** Changes to notification delivery mechanisms (e.g., upgrading to a new push notification API, switching SMS providers) are isolated to the Notification Delivery Service. Publishers — the core banking services — require zero changes. This reduces regression risk and allows faster iteration on the notification channel.

| Challenge                             | Pub/Sub Solution                                  |
|---------------------------------------|---------------------------------------------------|
| Notification service temporarily down | Messages persist in broker; processed on recovery |

| Challenge                              | Pub/Sub Solution                                        |
|----------------------------------------|---------------------------------------------------------|
| Broker crashes                         | Disk-persisted messages survive; no data loss           |
| Message fails after multiple retries   | Moved to Dead-Letter Queue for manual inspection        |
| High transaction volume spike          | Broker buffers messages; subscriber scales horizontally |
| Need to add a new notification channel | Deploy new subscriber; no publisher changes needed      |
| One subscriber instance fails          | Broker reassigns its messages to healthy instances      |